

Compte rendu projet MediaTek86

Contexte du projet

Dans le cadre de l'atelier de professionnalisation du BTS SIO option SLAM, le projet MediaTek86 avait pour objectif de développer une application permettant de gérer le personnel et les absences d'une entreprise fictive.

L'application a été réalisée en C# WinForms avec une base de données MySQL et une architecture MVC.

Le projet organisé en six étapes, devait permettre :

- La gestion du personnel
- La gestion des absences
- La sécurisation de l'accès à l'application
- La création d'un installateur
- La documentation complète du projet

Étape 1 – Préparation de l'environnement et création de la base de données

Préparation de l'environnement de travail

Avant de commencer le développement, l'environnement de travail a été préparé afin de disposer de tous les outils nécessaires.

Les actions suivantes ont été réalisées :

- création de la solution MediaTek86 ;
- préparation de l'arborescence du projet ;
- configuration de Git et GitHub.

Cette étape a permis de travailler dans un environnement stable et organisé.

Création de la base de données

La base de données a été créée à partir du schéma conceptuel de données fourni dans le dossier documentaire.

Les tables suivantes ont été créées :

- personnel
- absence
- service
- motif
- responsable

Les relations entre les tables ont ensuite été mises en place afin d'assurer la cohérence des données.

Des données de test ont également été ajoutées.

Sécurisation de l'accès à la base

L'accès à la base de données a été sécurisé grâce à la création d'un utilisateur MySQL disposant des droits nécessaires.

La table **responsable** a été utilisée pour gérer l'authentification dans l'application.

Les éléments suivants ont été mis en place :

- ajout d'un login
- ajout d'un mot de passe
- hashage du mot de passe avec SHA2
- vérification des identifiants lors de la connexion.

Cette étape permet de limiter l'accès à l'application aux utilisateurs autorisés.

Étape 2 – Structuration de l'application et création du visuel

Étude du dossier documentaire

Le dossier documentaire fourni a été étudié afin de comprendre les besoins du projet.

Les documents analysés comprenaient :

- le diagramme de cas d'utilisation
- le descriptif des fonctionnalités

Cette analyse a permis d'identifier les différentes fonctionnalités à développer.

Mise en place de l'architecture MVC

L'application a été organisée selon une architecture MVC afin de séparer les différentes parties du projet : les vues ,les contrôleurs, les modèles et les accès aux données.

Les packages ont été créés en respectant une structure proche de l'application **Habilitations**.

Création du dépôt GitHub

Afin de sauvegarder le projet et suivre les différentes étapes du développement, un dépôt GitHub a été créé <https://github.com/sadeqwarda/MediaTek86>

Les actions suivantes ont été réalisées :

- initialisation Git
- première sauvegarde du projet
- création de commits réguliers
- synchronisation avec GitHub

Mise en place du Kanban

Un GitHub Project de type Kanban a été créé afin d'organiser les tâches du projet.

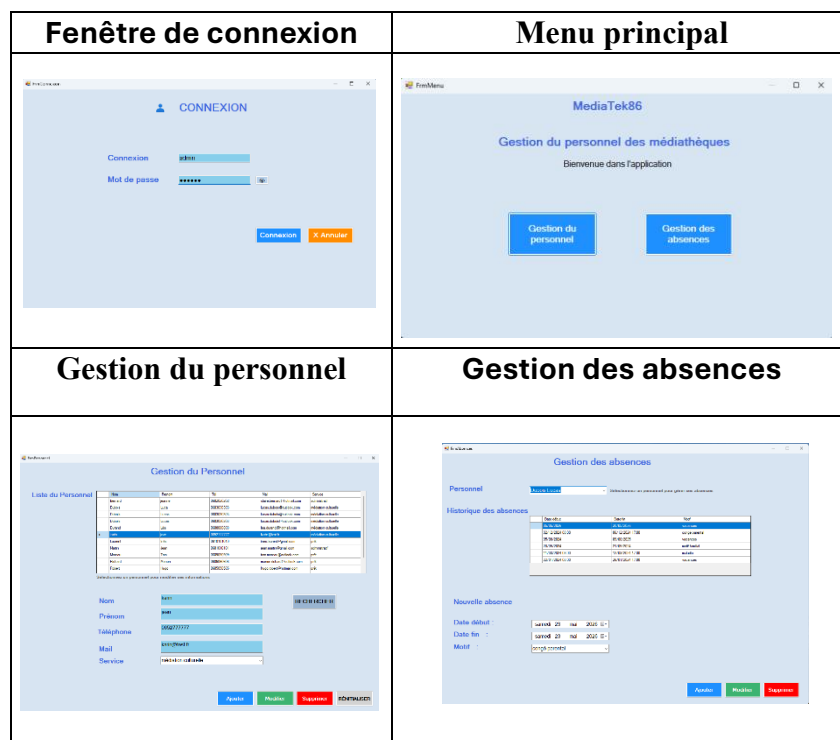
Les différentes fonctionnalités ont été ajoutées sous forme d'issues.

Le Kanban a été mis à jour progressivement pendant le développement.

Création des interfaces graphiques

Les interfaces de l'application ont été créées avec WinForms.

Les fenêtres suivantes ont été développées :



Les composants graphiques ont été organisés afin d'obtenir une interface simple et lisible.

Étape 3 – Développement de l'authentification et du menu

Développement de la connexion

La fenêtre de connexion a été développée dans la classe **FrmConnexion**.

Cette fenêtre permet :

- la saisie du login
- la saisie du mot de passe
- le contrôle des identifiants
- l'ouverture du menu principal

La méthode **btnConnexion_Click()** récupère les informations saisies puis vérifie les identifiants dans la base de données.

Un bouton permettant d'afficher ou masquer le mot de passe a également été ajouté grâce à la méthode **btnAfficherPwd_Click()**.

Le contrôle des identifiants s'effectue à partir de la table responsable.

Création du menu principal

Le menu principal a été développé dans la classe **FrmMenu**.

Il permet d'accéder aux différentes fonctionnalités de l'application.

Les méthodes **btnPersonnel_Click()** et **btnAbsences_Click()** permettent d'ouvrir respectivement la gestion du personnel et la gestion des absences.

Étape 4 – Développement de la gestion du personnel

Affichage des employés

La gestion du personnel a été développée dans la classe **FrmPersonnel**.

Les employés sont affichés dans un **DataGridView** contenant : le nom, le prénom, le téléphone, le mail et le service.

La méthode **RemplirListePersonnel()** permet de charger les employés depuis la base de données.

Lorsqu'un personnel est sélectionné, la méthode **SelectionnerPersonnel()** affiche automatiquement ses informations dans les champs du formulaire.

Ajout d'un personnel

La méthode **btnAjouter_Click()** permet d'ajouter un nouvel employé dans la base de données.

Plusieurs contrôles ont été ajoutés :

- vérification des champs obligatoires ;
- contrôle des doublons ;
- validation des données.

Modification et suppression

La méthode **btnModifier_Click()** permet de mettre à jour les informations d'un personnel existant.

La méthode **btnSupprimer_Click()** permet de supprimer un personnel après confirmation de l'utilisateur.

Recherche et ergonomie

Une barre de recherche a été ajoutée afin de filtrer rapidement les employés.

Étape 5 – Développement de la gestion des absences

Gestion des absences

La gestion des absences a été développée dans la classe **FrmAbsences**.

Une fenêtre dédiée permet d'afficher les absences du personnel sélectionné.

Les informations affichées comprennent : la date de début, la date de fin et le motif de l'absence.

La méthode **RemplirListeAbsences()** permet de charger les absences depuis la base de données.

Ajout, modification et suppression

La méthode **btnAjouter_Click()** permet d'ajouter une absence.

La méthode **btnModifier_Click()** permet de modifier une absence existante.

La méthode **btnSupprimer_Click()** permet de supprimer une absence après confirmation.

Les motifs d'absence sont sélectionnés à partir d'une liste déroulante.

Contrôle des dates

Des contrôles ont été ajoutés afin de garantir la cohérence des données :

- la date de début doit être inférieure ou égale à la date de fin ;
- une absence ne peut pas être enregistrée plusieurs fois avec la même date.

Contrôle anti-chevauchement

Un contrôle spécifique a été développé afin d'empêcher qu'un employé possède plusieurs absences sur des périodes qui se chevauchent.

Les méthodes **ChevauchementAjout()** et **ChevauchementModification()** permettent de vérifier les conflits de dates.

Exemple :

- absence existante : 01/06 → 05/06 ;
- nouvelle absence : 03/06 → 08/06.

Dans ce cas, l'application affiche un message d'erreur.

Ce contrôle fonctionne également lors de la modification d'une absence.

Étape 6 – Documentation et déploiement

Documentation du projet

Plusieurs documents ont été réalisés afin de documenter le projet : README GitHub , captures d'écran, MCD , diagramme de paquetages et compte rendu d'activité

Gestion du projet avec GitHub

Le projet a été sauvegardé régulièrement sur GitHub grâce à des commits explicites permettant de suivre les différentes étapes du développement.

Le Kanban a également été mis à jour au fur et à mesure de l'avancement.

Création de l'installateur

Un projet Setup a été créé afin de générer un installateur de l'application.

Les fichiers suivants ont été générés : fichier MSI, setup.exe et raccourci de l'application.

Des tests ont ensuite été réalisés afin de vérifier le bon fonctionnement de l'installation.

Organisation générale du projet

Architecture MVC

L'application a été développée en utilisant une architecture MVC (Modèle – Vue – Contrôleur).

Cette architecture permet de séparer les différentes parties du projet afin de faciliter :

Organisation des packages

Le projet est organisé en plusieurs packages :

- model : contient les classes métier
- view : contient les interfaces graphiques
- controller : contient les traitements et la logique applicative

- dal : contient les accès aux données
- bddmanager : gère la connexion MySQL.

DAL et BddManager

Les classes DAL permettent la communication entre l'application et la base de données.

Exemples :

- PersonnelAccess ;
- AbsenceAccess ;
- ResponsableAccess.

Ces classes utilisent des requêtes SQL sécurisées via la classe BddManager, qui centralise la connexion à la base de données.

Difficultés rencontrées

Gestion du contrôle anti-chevauchement

La principale difficulté concernait la gestion des absences, lors de la modification d'une absence, l'application détectait parfois un faux chevauchement avec l'absence en cours de modification.

Plusieurs corrections ont été nécessaires afin :

- de comparer correctement les dates
- d'ignorer l'absence sélectionnée
- de différencier l'ajout et la modification

Gestion des erreurs MySQL

Des erreurs liées aux clés primaires sont apparues lors de l'ajout d'absences avec une même date de début.

Des contrôles supplémentaires ont été ajoutés afin :

- d'éviter les doublons
- d'empêcher les erreurs SQL
- d'améliorer les messages affichés à l'utilisateur

Synchronisation GitHub

Des difficultés ont également été rencontrées lors de la synchronisation entre le dépôt local et GitHub.

Il a fallu :

- mettre à jour les fichiers installateur

- gérer les commits
- synchroniser correctement les modifications

Gestion de l'installateur

La mise à jour de l'installateur nécessitait plusieurs étapes :

- compilation en mode Release
- régénération du projet Setup
- remplacement des fichiers MSI
- réinstallation de l'application

Conclusion

Le projet **MediaTek86** m'a permis de mettre en pratique les compétences acquises durant la formation BTS SIO SLAM dans un contexte proche du milieu professionnel.

Ce projet m'a permis de renforcer mes compétences en :

- développement C# WinForms
- architecture MVC
- gestion de bases de données MySQL
- utilisation de Git et GitHub
- documentation technique
- déploiement d'applications

Cette expérience m'a également permis de mieux comprendre les différentes étapes du développement d'une application, depuis l'analyse du besoin jusqu'au déploiement final.